Artificial Neural Networks

April 25, 2024

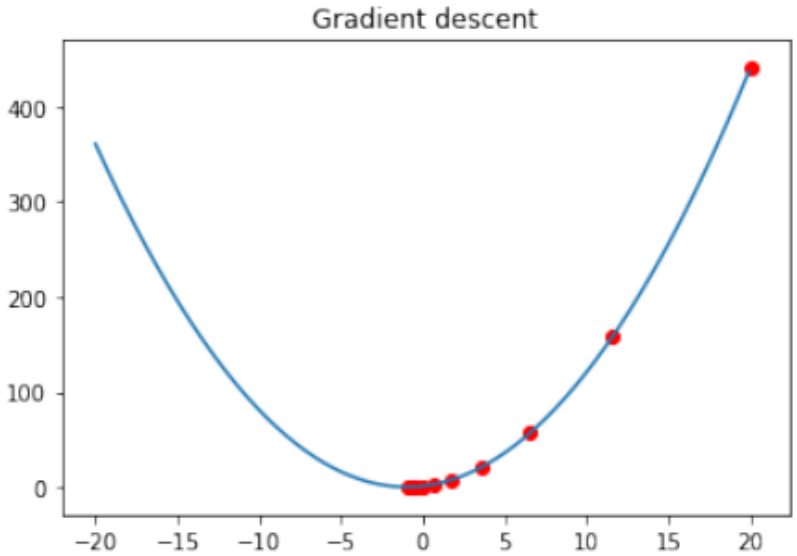
Linear vs nonlinear

Artificial Neural networks

Recomandari legate de structura

Training algorithm

Gradient Descent - reamintire



w $\leftarrow$ any point in the parameter space

loop until convergence **do**

  for each w_i **in w do**

$$w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} Loss(\mathbf{w})$$

- Algoritm de gasire a minimului unei functii
- Linear regression
- Logistic regression

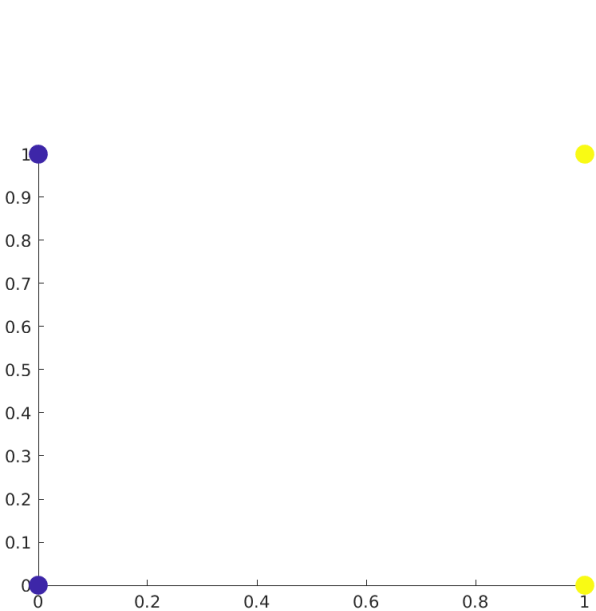
Linear vs nonlinear

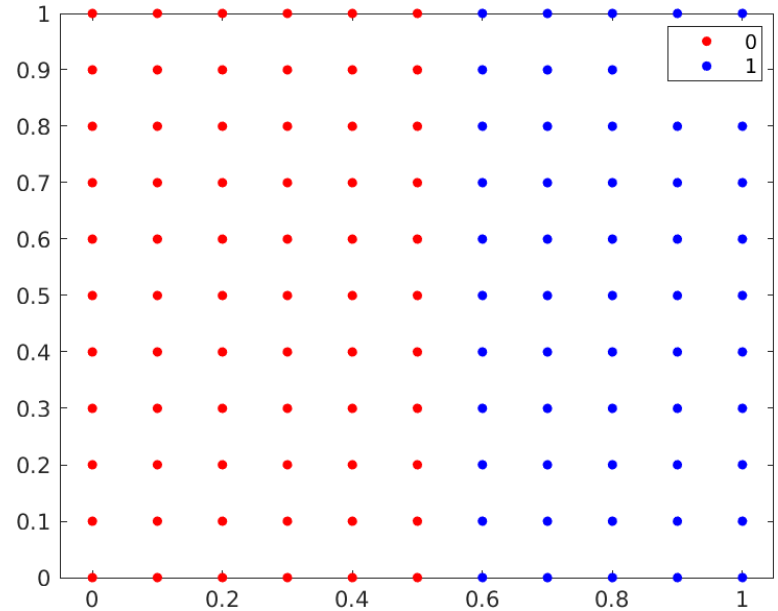
Artificial Neural networks

Recomandari legate de structura

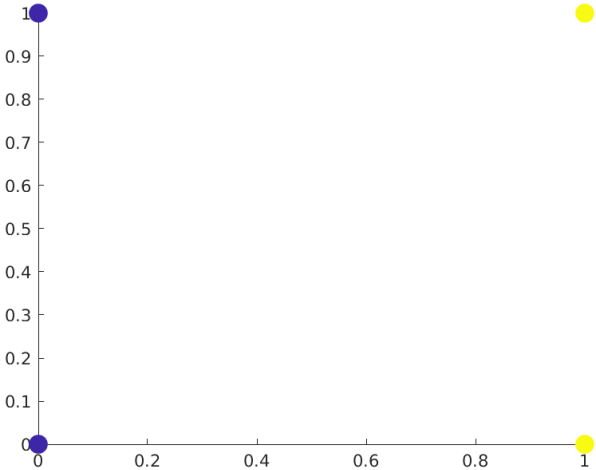
Training algorithm

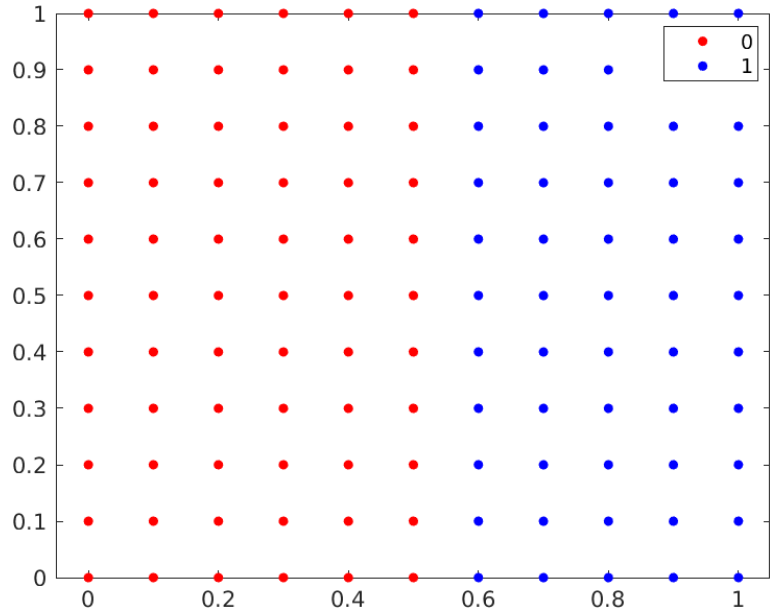
Date care nu sunt liniar separabile

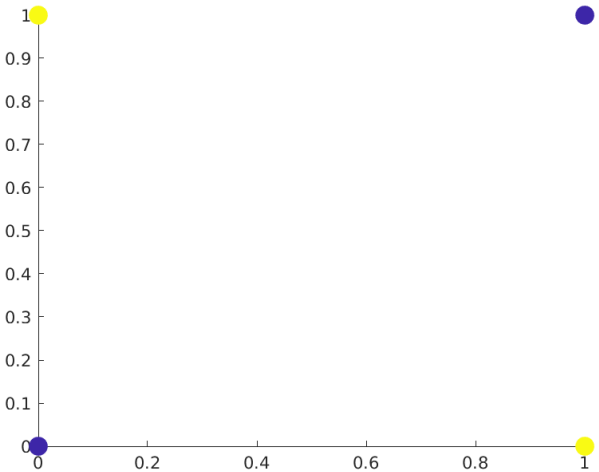




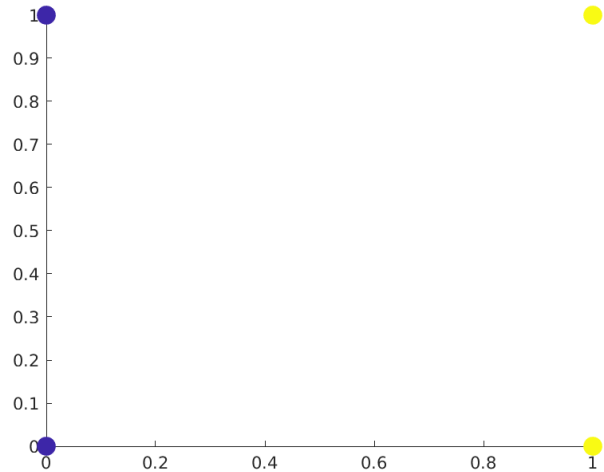
Date care nu sunt liniar separabile

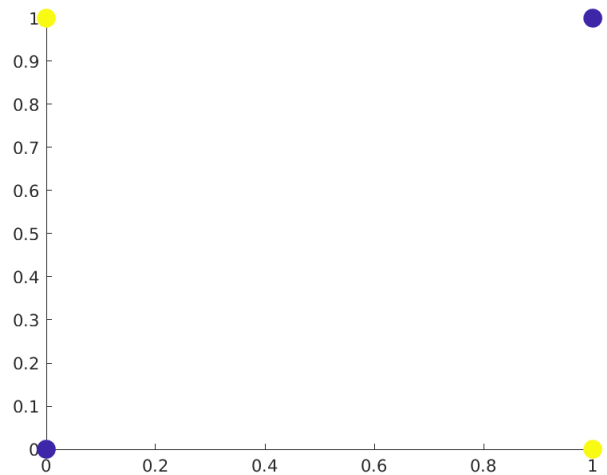


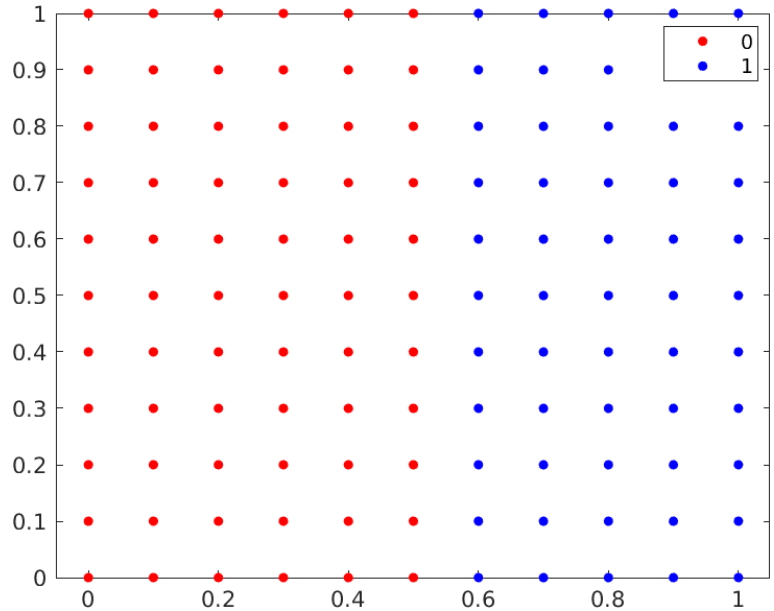


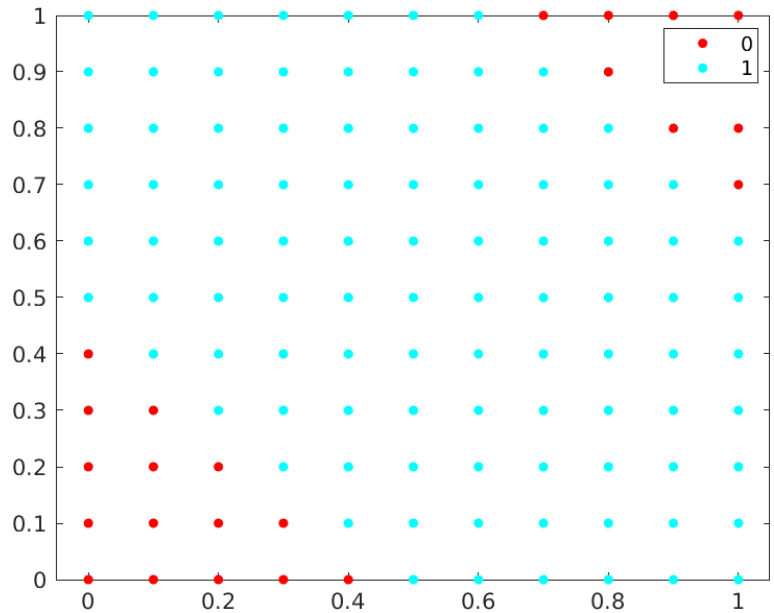


Date care nu sunt liniar separabile



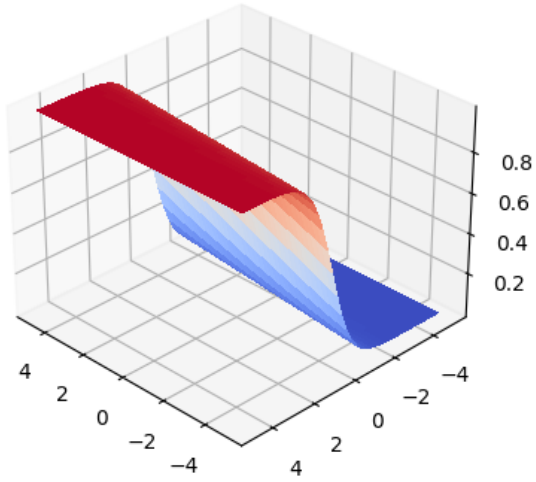




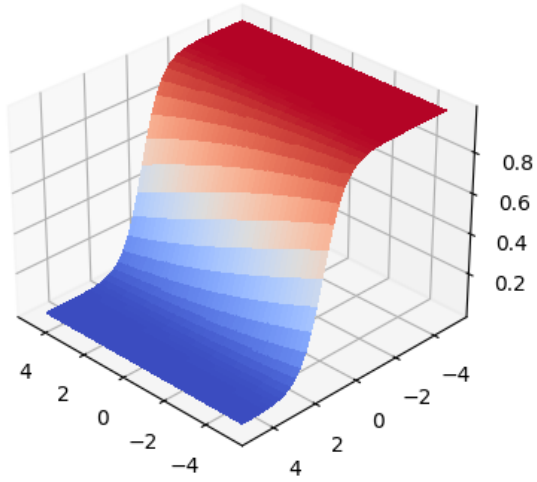


see notebook

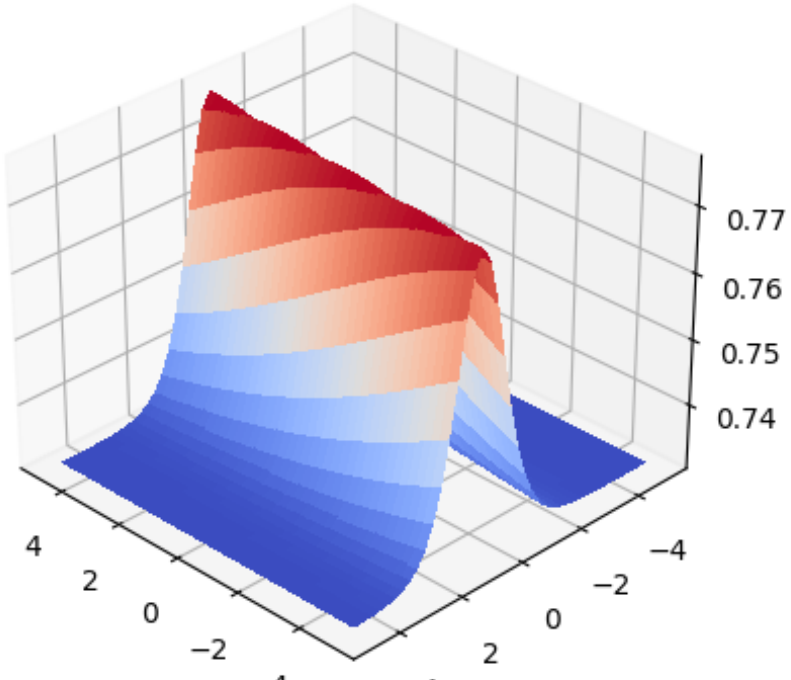
Perceptron - soft threshold separation

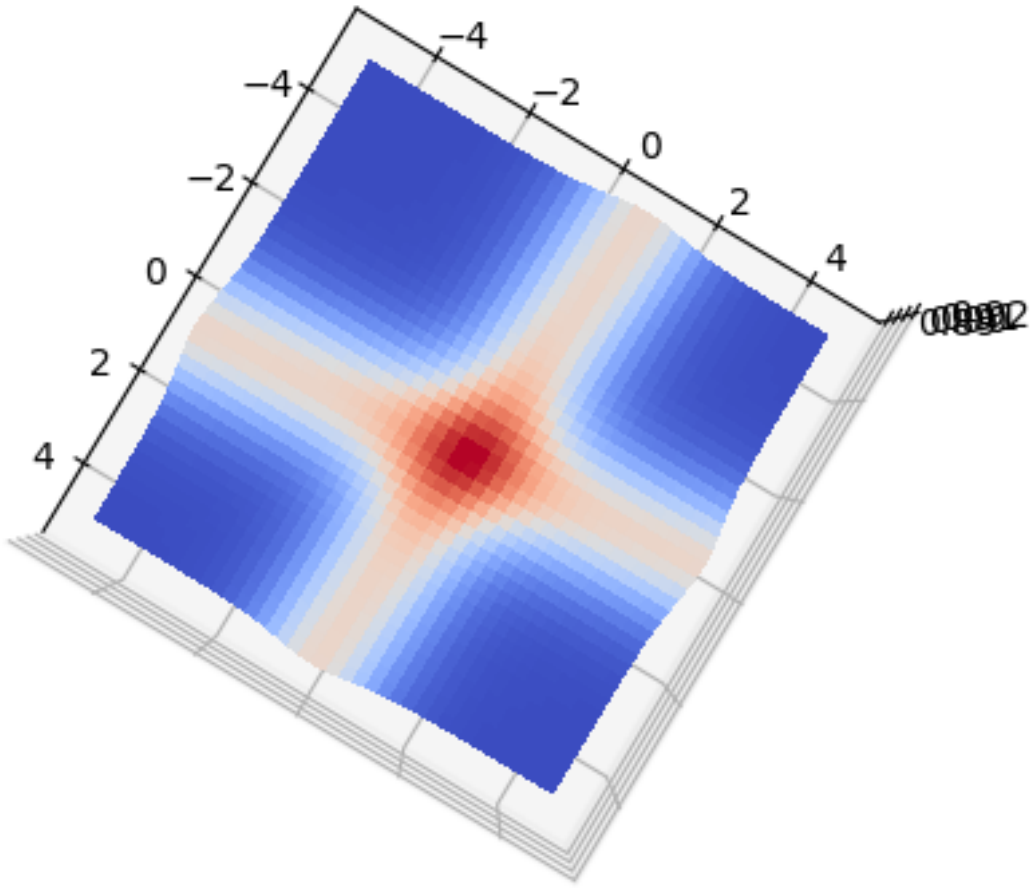


Opposite-facing soft threshold separation

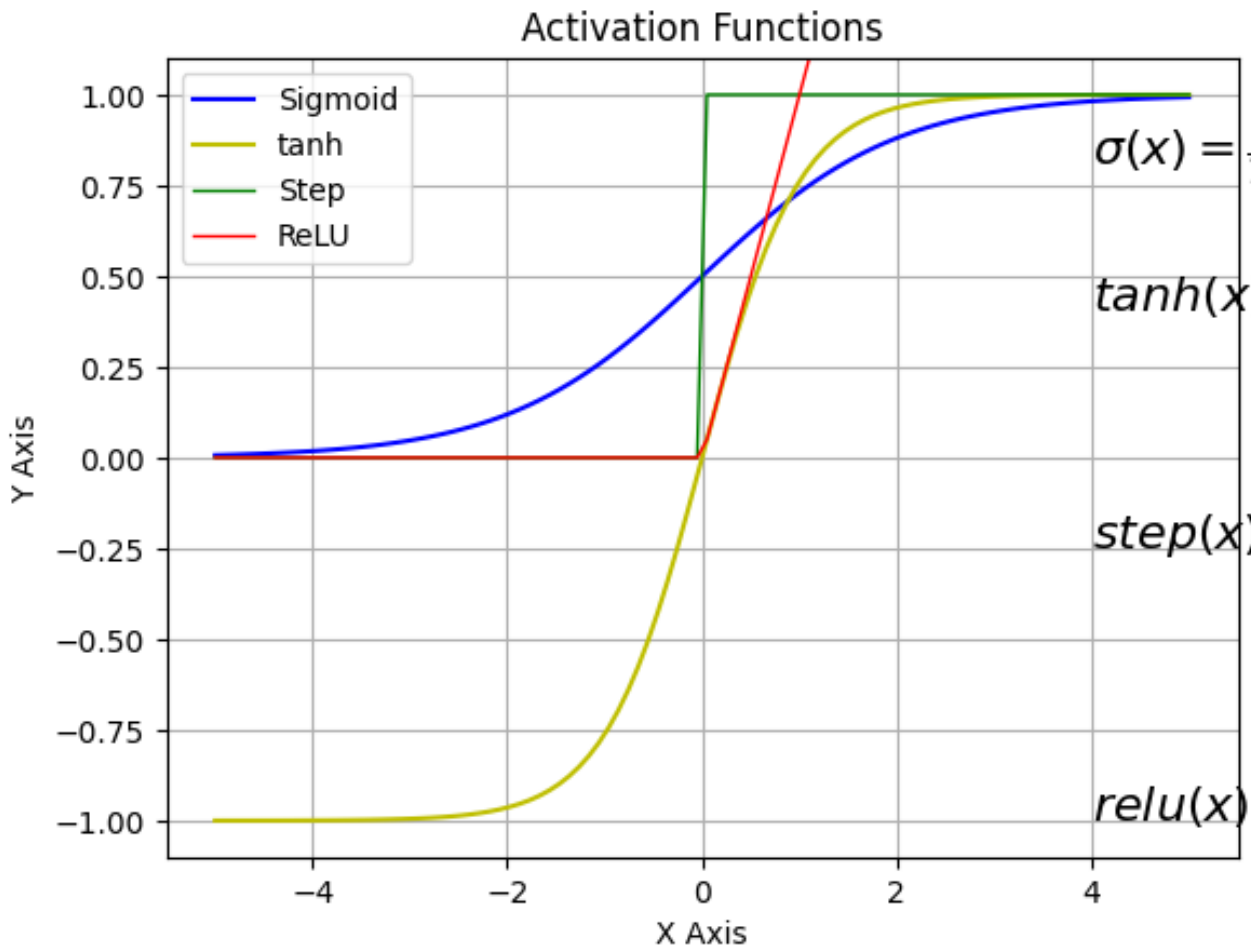


Sum of two opossite-facing soft thresold and thresholding the result




−4
−2
0
2
4
−4
−2
0
2
4

Activation Functions



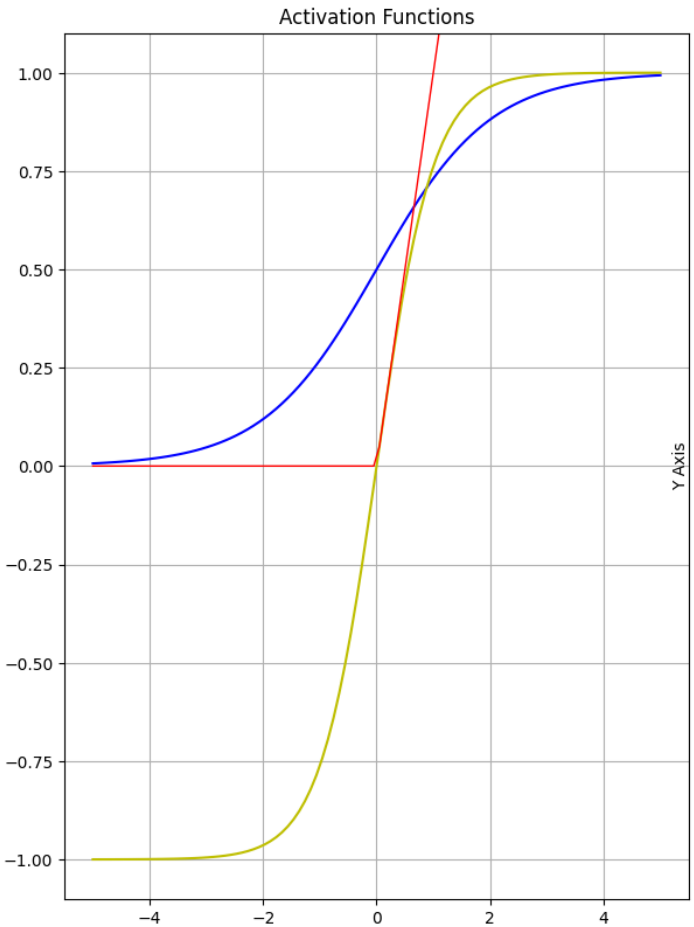
$$\sigma(x) = \frac{1}{1+e^{-x}}$$

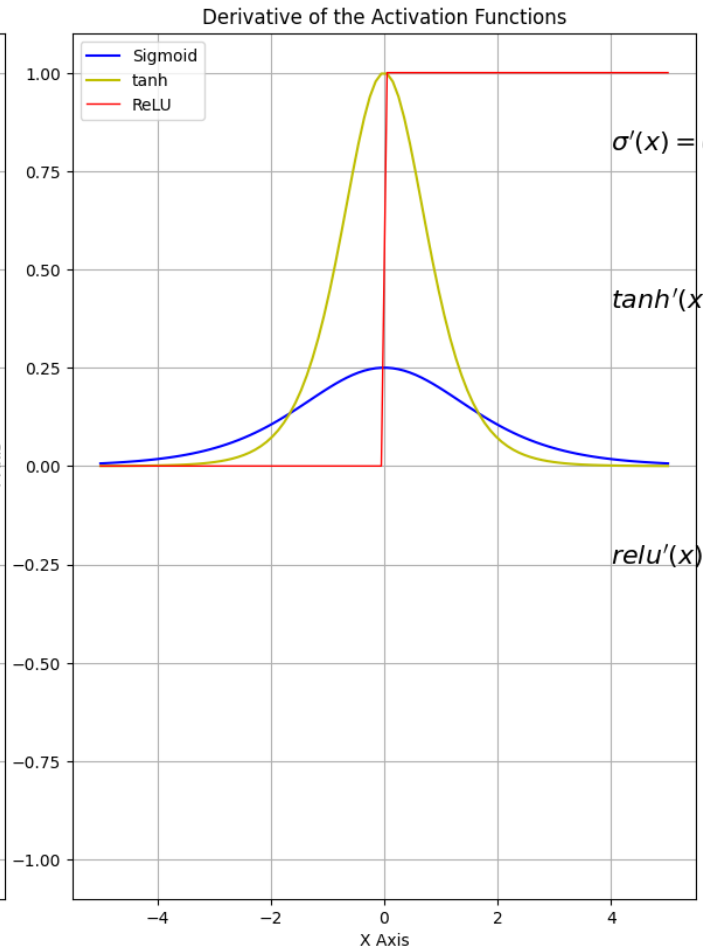
$$tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$$

$$step(x) = \begin{cases} 0, & x < 0 \\ 1, & x \geq 0 \end{cases}$$

$$relu(x) = \begin{cases} 0, & x < 0 \\ x, & x \geq 0 \end{cases}$$

Activation Functions - derivatele



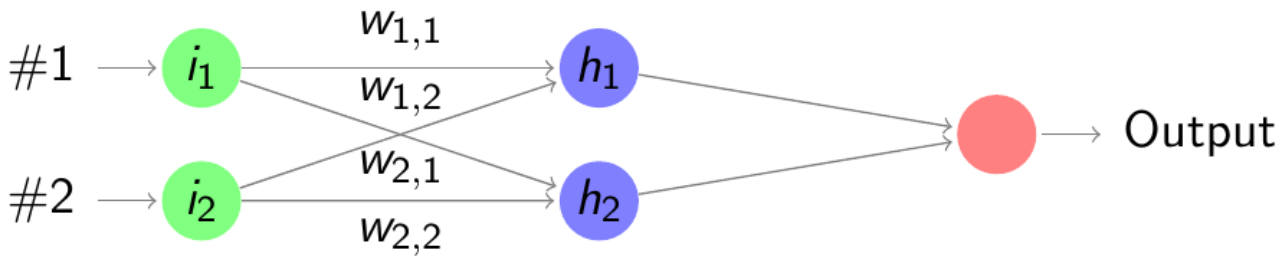


$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

$$tanh'(x) = 1 - tanh^2(x)$$

$$relu'(x) = \begin{cases} 0, & x < 0 \\ 1, & x \geq 0 \end{cases}$$

(Feed Forward) Neural networks



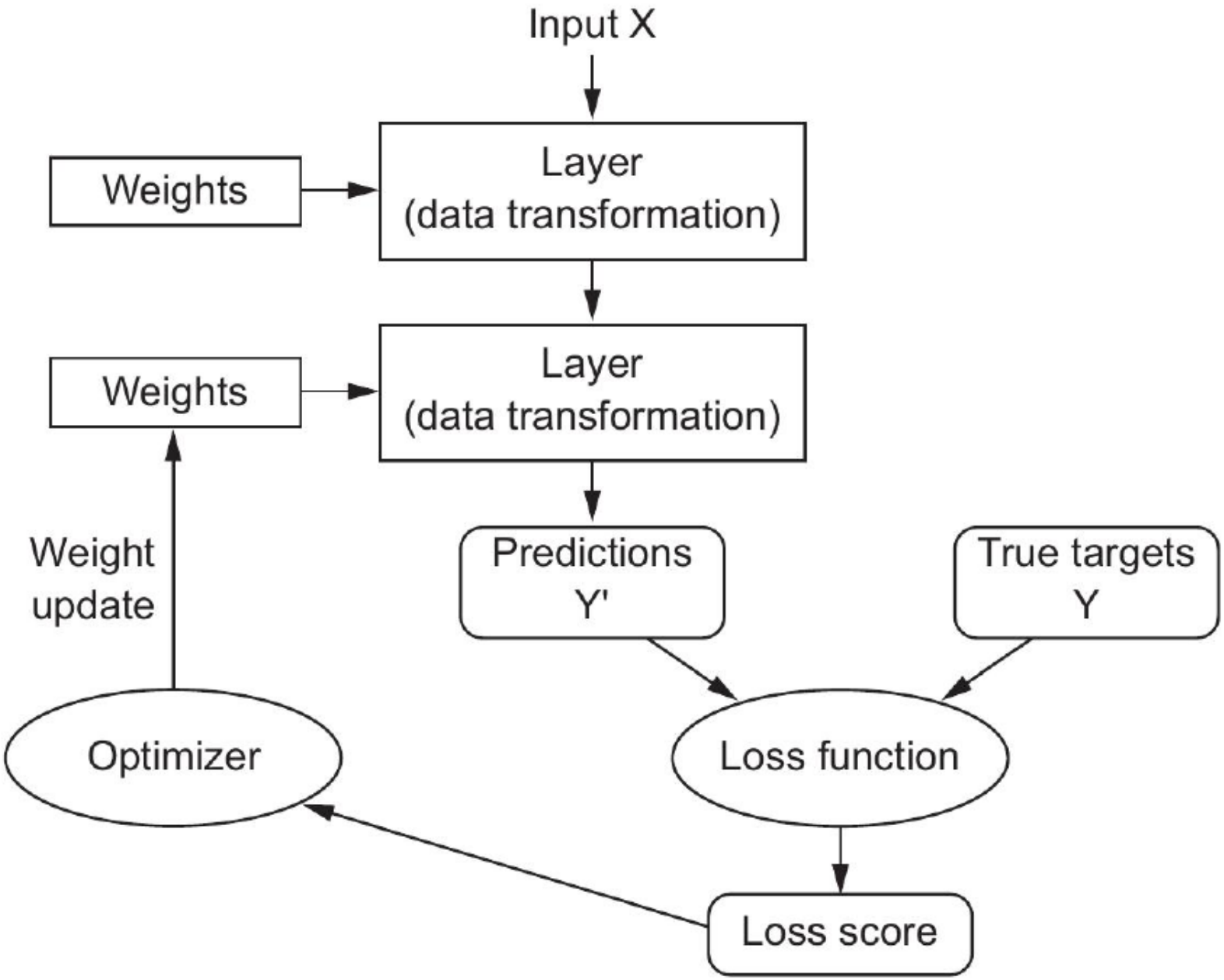
$$a_j = g(in_j)$$

$$a_j = g(\sum_i (w_{i,j} * a_i))$$

- a_i output of neuron i,
- $a_0 = 1$ bias
- $W_{i,j}$ weight between neuron i and neuron j
- g activation function

example: $a_{h_1} = g(w_{1,1} * a_i + w_{2,1} * a_2 + w_{0,1})$

Neural networks



Basic Loss functions

- Pt regresie

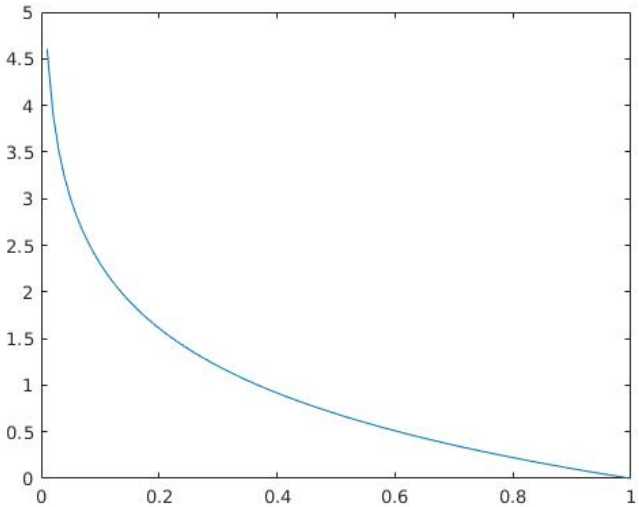
$$MSE = \frac{1}{m}\sum_{i=1}^{m}(y - \hat{y})^2$$

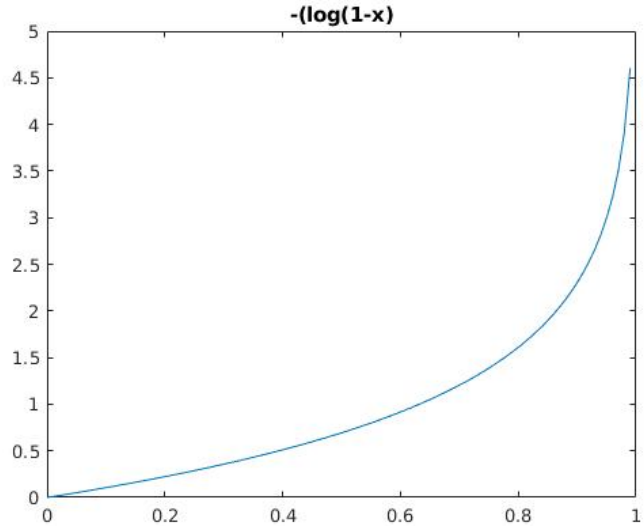
- Pt clasificare binara - Log loss, cross entropy

$$LogLoss = \frac{1}{m}\sum_{i=1}^{m}[-y\ log(\hat{y}) - (1-y)\ log(1-\hat{y})]$$

Classification Loss - reamintire

clasificare folosind o singura probabilitate:





- Clasa 1: predictie prob 0.6, resp 0.8
- Clasa 0: predictie prob 0.2, resp 0.8

see Losses notebook

Tipuri de layere - exemple

- Input Layers: image, sequence, features
- Output Layers: regression, classification
- Convolutional Layers
- FullyConnectedLayer - un neuron e conectat la toti neuronii din layerul anterior
- Sequence (recurrent) layers
- ActivationLayers
- Normalization, Dropout, Pooling, Combination

Exemple de arhitecturi 1

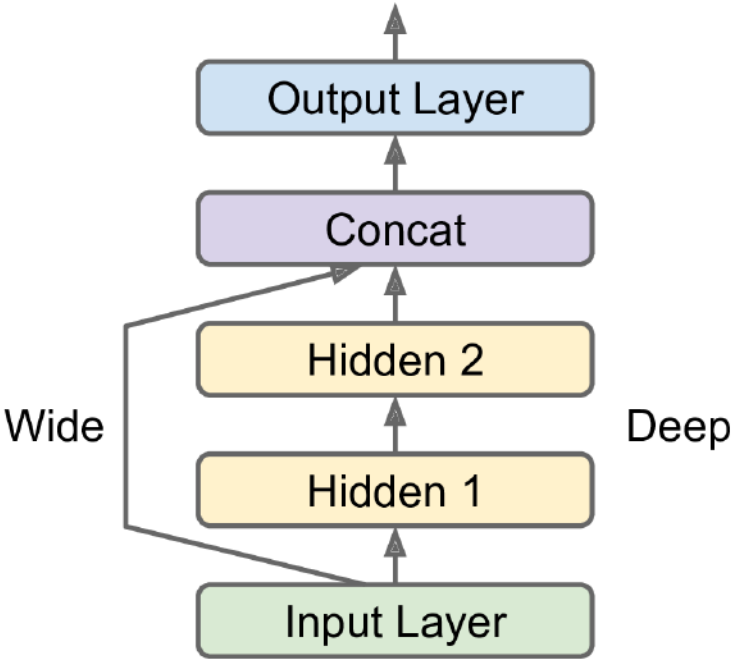


Figure 10-14. Wide & Deep neural network

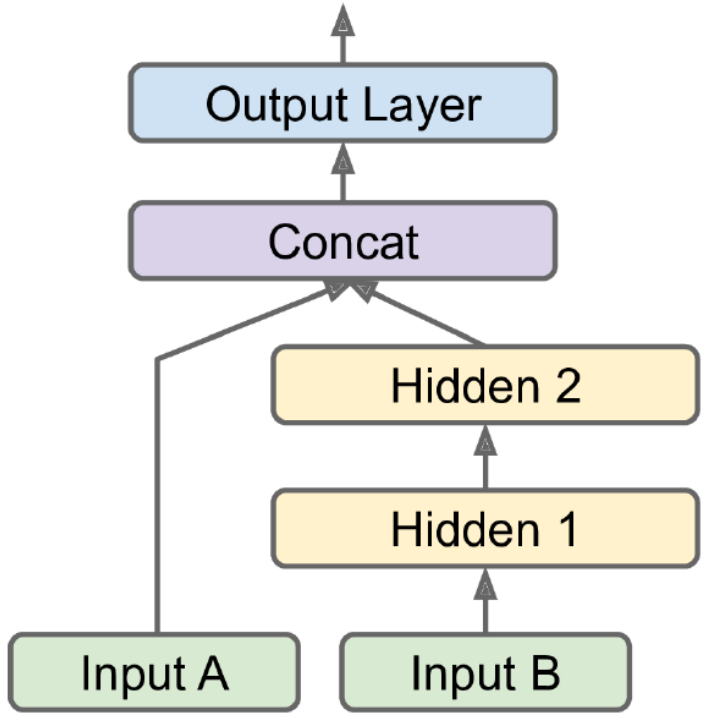


Figure 10-15. Handling multiple inputs

Exemple de arhitecturi 2

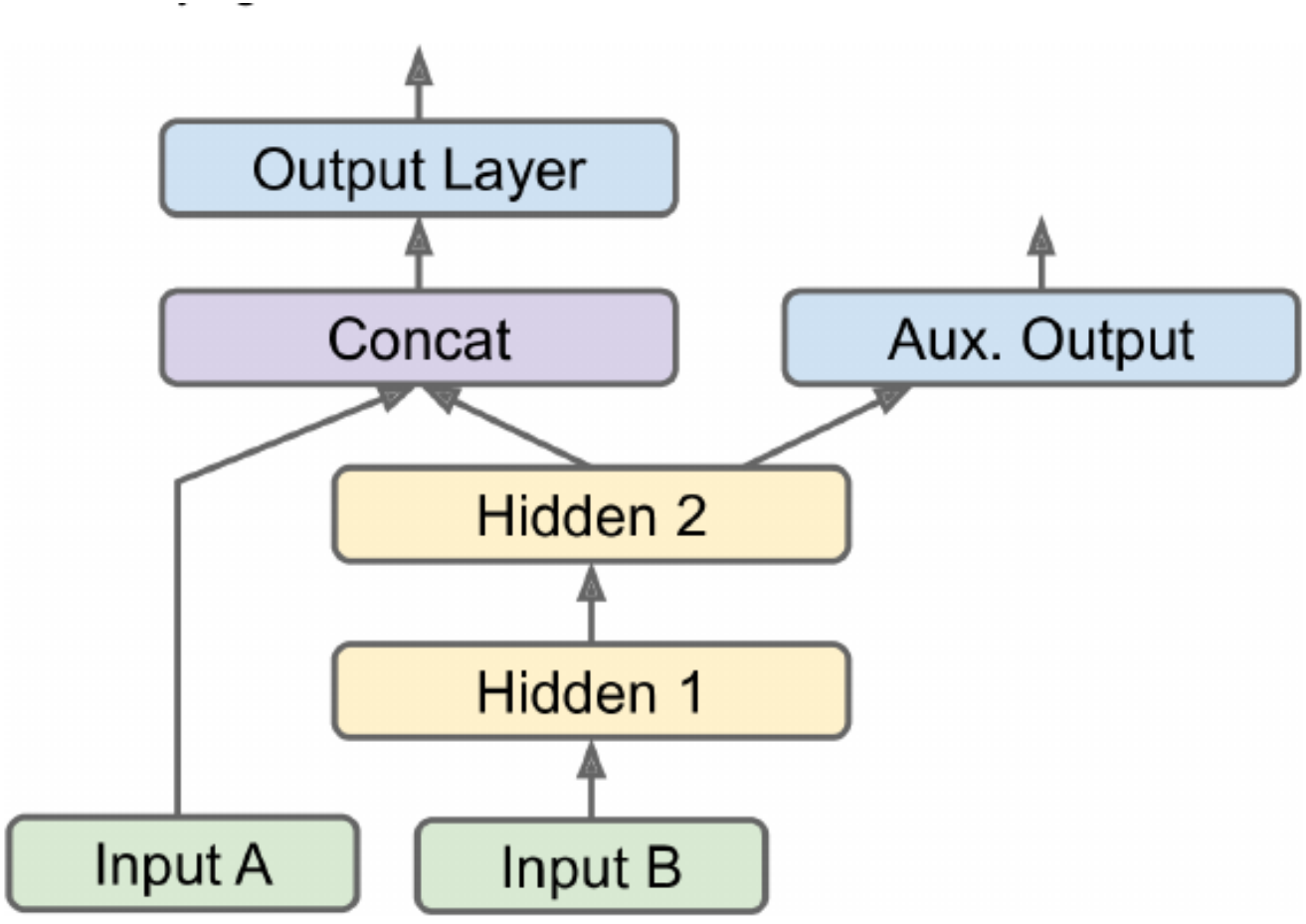


Figure 10-16. Handling multiple outputs, in this example to add an auxiliary output for regularization

Linear vs nonlinear

Artificial Neural networks

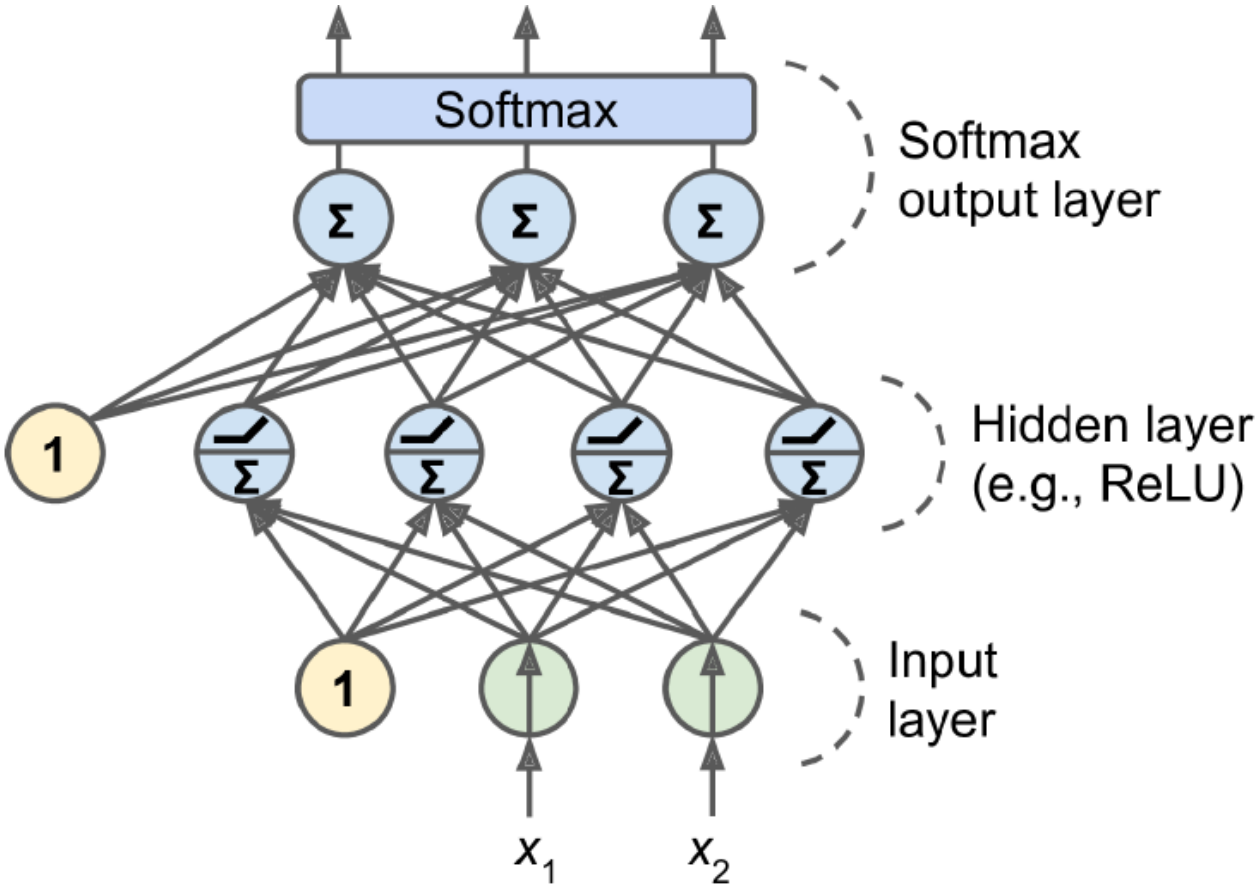
Recomandari legate de structura

Training algorithm

NN pt regresie

Hyperparameter	Typical values
#input neurons	one per input feature
#hidden layers	depend on the problem, but typically 1 to 5
# neurons per hidden layer	depend on teh problem, but typically 10 to 100
#output neurons	1 per prediction dimension
hidden activation	relu or selu
output activation	none
loss function	MSE or MAE(for outliers)

NN pt clasificare cu mai mult de 2 clase



Functia softmax (normalized exponential) - generalizare a functiei logistic

$$\sigma(\mathbf{z}_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

NN for classification

Hyper-parameters	Binary classification	Multilabel binary classification	Multiclass classification
input and hidden layers	same as regression	same as regression	same as regression
#outputs	1	1 per label	1 per class
outputlayer activation	logistic	logistic	softmax
loss function	crossentropy	crossentropy	crossentropy

Linear vs nonlinear

Artificial Neural networks

Recomandari legate de structura

Training algorithm

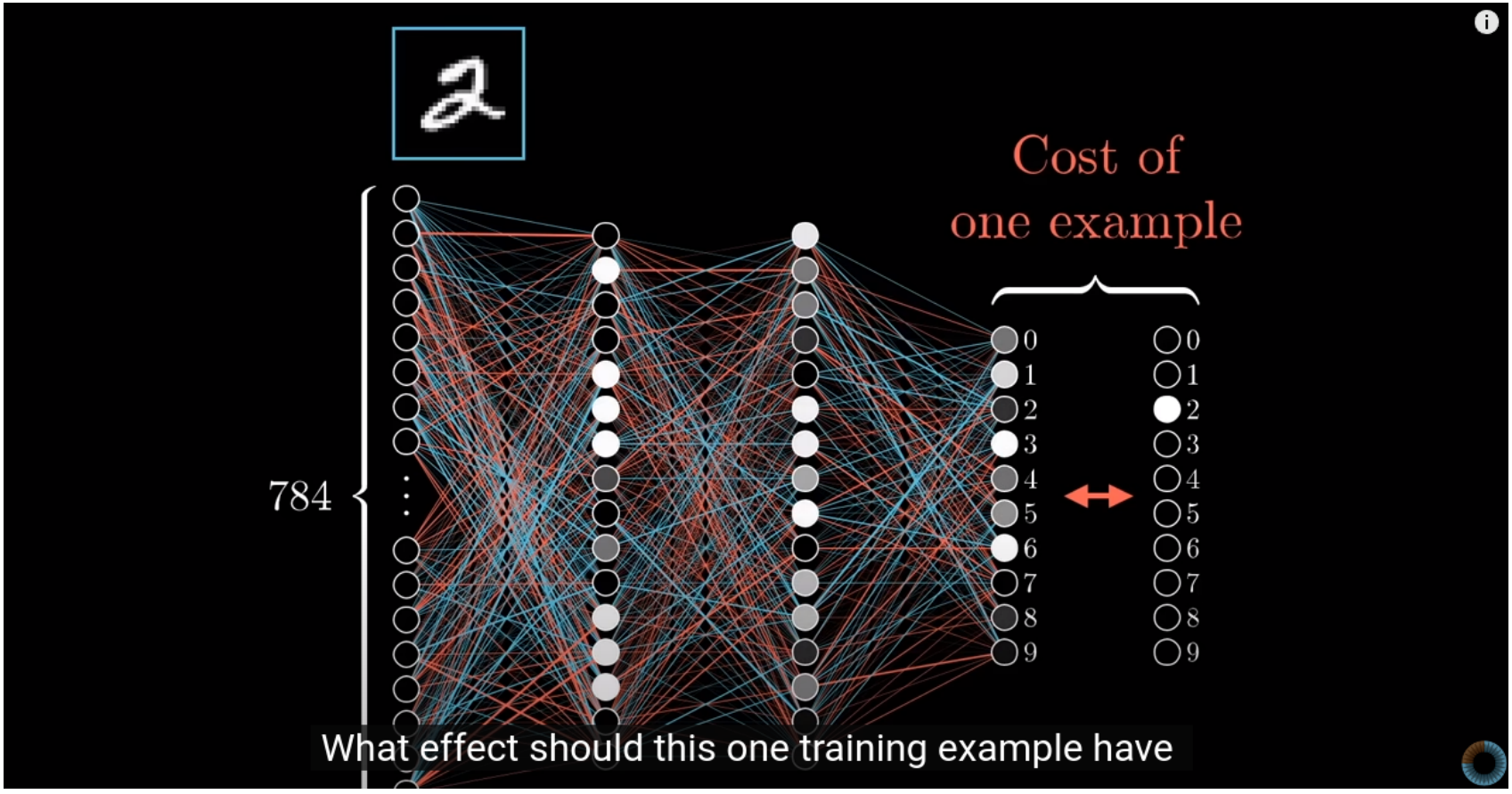
Training NN - Backpropagation

Minimizarea loss-ului - algoritmi de optimizare (e.g. gradient descent)
Pentru fiecare minibatch:

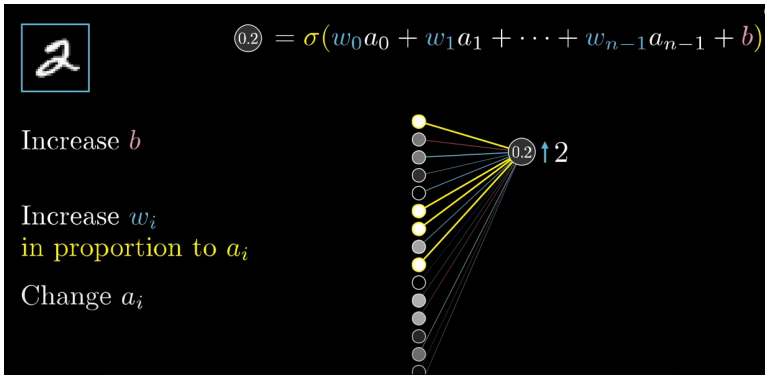
1. Feed forward - se calculeaza iesirea pt fiecare neuron
2. Backpropagate the error - se modifica w folosind GradientDescent sau alt algoritm de optimizare

O epoca - folosirea tuturor exemplelor

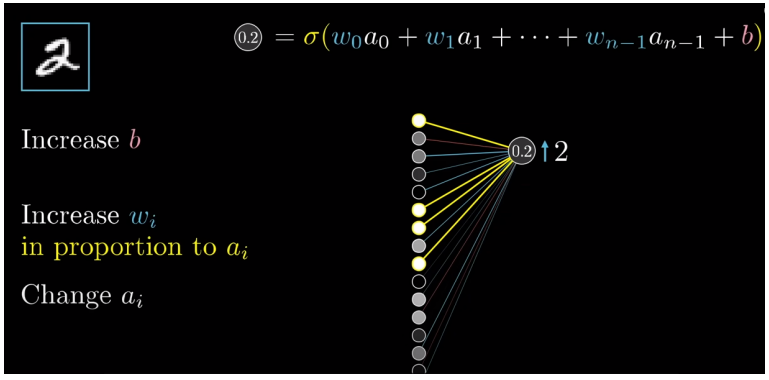
Backpropagation intuition

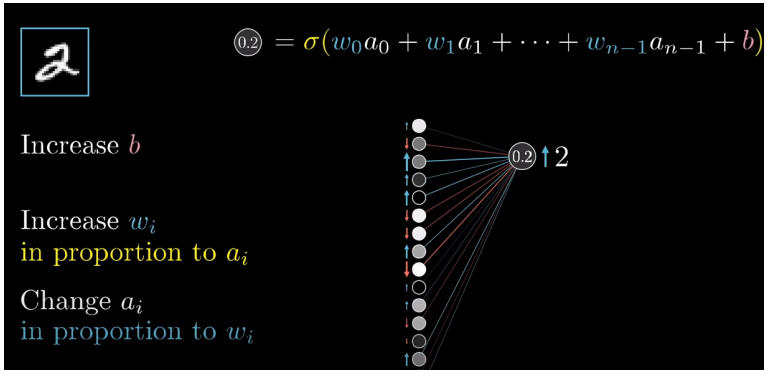


Cum putem reduce eroarea?

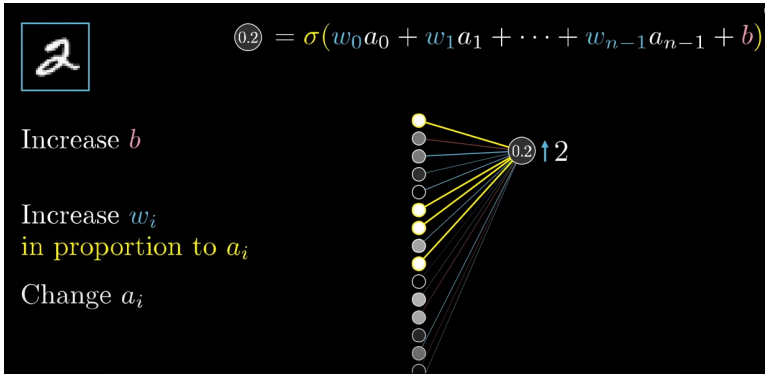


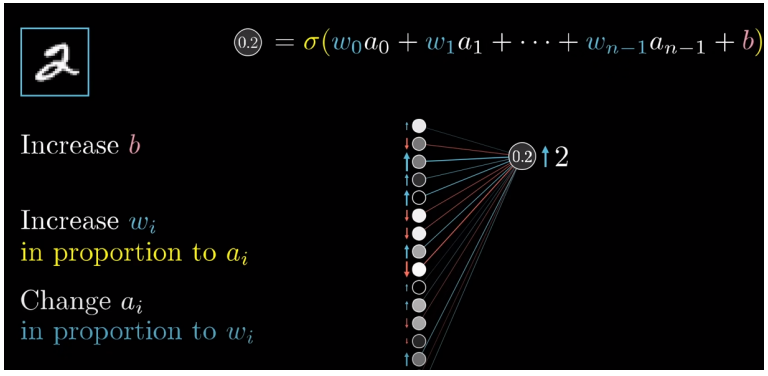
Cum putem reduce eroarea?

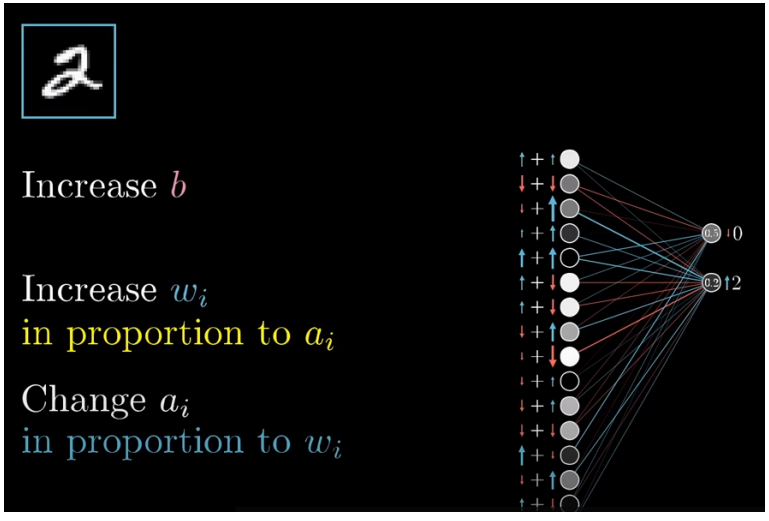




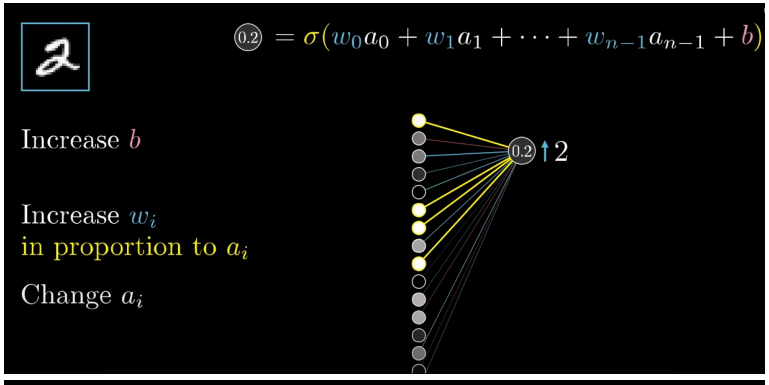
Cum putem reduce eroarea?

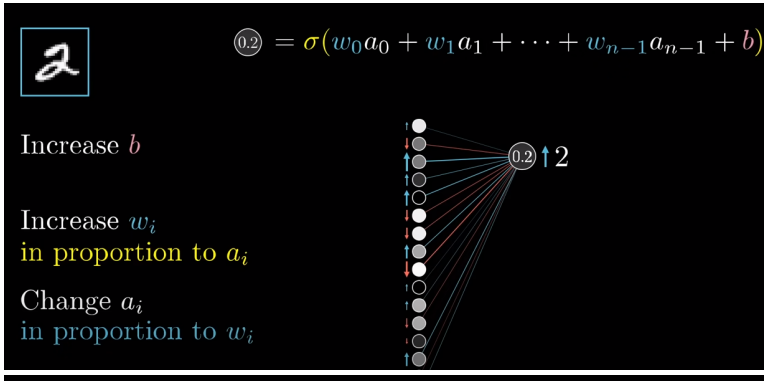


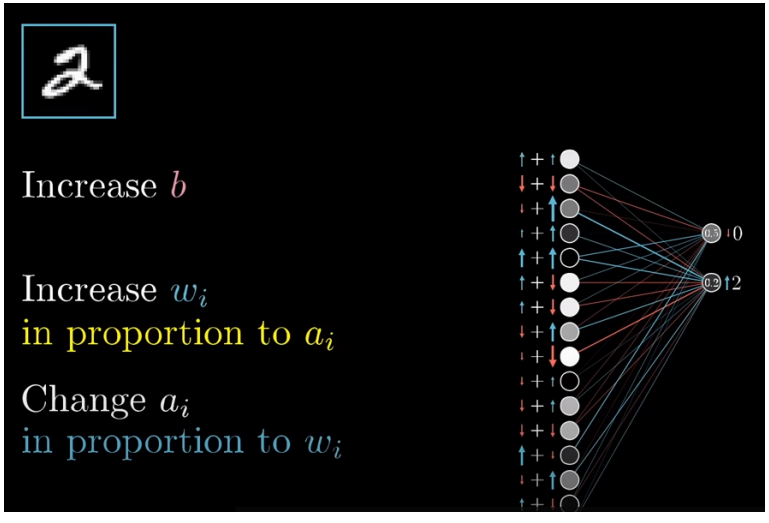


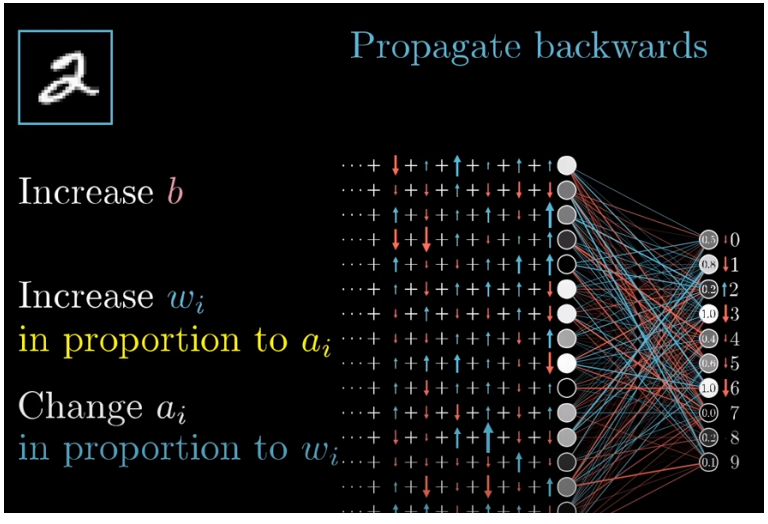


Cum putem reduce eroarea?







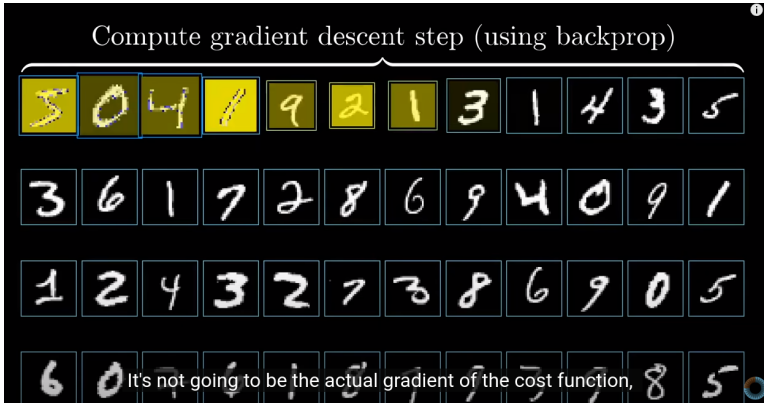


Minibatch


Average over
all training data
w_0 −0.08 +0.02 −0.02 +0.11 −0.05 −0.14 ··· −0.08
w_1 −0.11 +0.11 +0.07 +0.02 +0.09 +0.05 ··· +0.12
w_2 −0.07 −0.04 −0.01 +0.02 +0.13 −0.15 ··· −0.06

Minibatch


Average over all training data
w_0 −0.08 +0.02 −0.02 +0.11 −0.05 −0.14 ··· → −0.08
w_1 −0.11 +0.11 +0.07 +0.02 +0.09 +0.05 ··· → +0.12
w_2 −0.07 −0.04 −0.01 +0.02 +0.13 −0.15 ··· → −0.06


Compute gradient descent step (using backprop)
It's not going to be the actual gradient of the cost function,

see Backpropagation 3Blue1Brown

Back-propagation

- Output layer

$$w_{j,i} = w_{j,i} + \alpha * a_j * \Delta_i$$

, unde $\Delta_i = Error_i * g'(in_i)$

- Hidden layer (k node - j node - i nodes)
  Propagated error:

$$\Delta_j = g'(in_j) \sum_i w_{j,i} \Delta_i$$

  - hidden node j is responsible for some fraction of the error Δ_i in each of the output nodes to which it connects; stronger connection (higher weight) means higher contribution to error

$$w_{k,j} = w_{k,j} + \alpha * a_k * \Delta_j$$

NN Playground

What can go wrong

- Vanishing gradient
- Exploding gradient
- Overfitting/underfitting
- Adjust size of the network
- Use regularization: L1, L2
- Dropout
- Early stopping
- Adjust learning rate

Vanishing/Exploding gradient

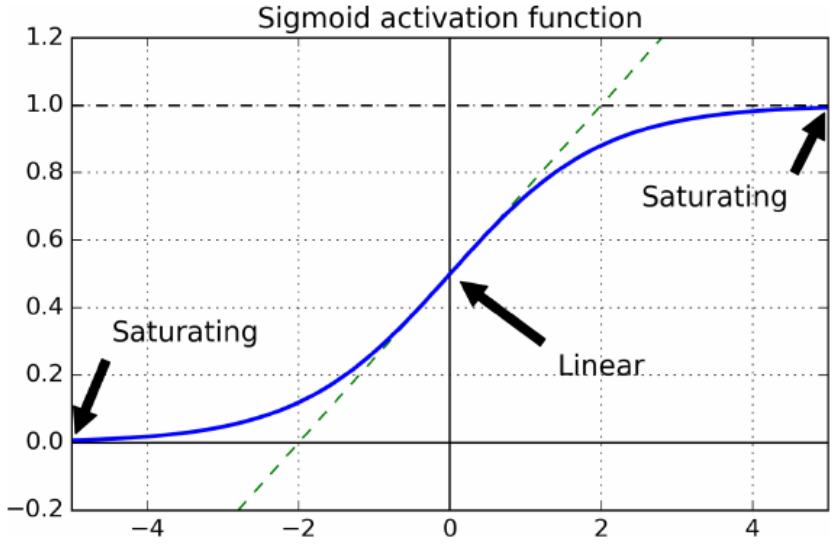


Figure 11-1. Logistic activation function saturation

Vanishing gradient - scaderea gradientilor catre layerele de inceput $\Rightarrow$ GD nu mai actualizeaza ponderile din acele layere; apare in special cand logistic function e folosit ca si functie de activare
Exploding gradient - cresterea gradientilor $\Rightarrow$GD diverge
Solutii 1: folosirea altor functiid e activare decat logistic(sigmoid) + scheme de initializare (2010 Xavier Glorot, Yoshua Bengio)

Solutii 2: Batch normalization

Adaugarea inainte sau dupa functia de activare a nivelelor ascunse
a: centrare la 0 si normalizare + scalare si shiftare $\Rightarrow$ 4 noi parametrii per input
Modelul invata valorile de shiftare si scalare optime
Se reduce numarul de epoci necesare pt invatare, insa creste complexitatea modelului (fiecare epoca dureaza mai mult cu BN)

1. $$\boldsymbol{\mu}_B = \frac{1}{m_B}\sum_{i=1}^{m_B}\mathbf{x}^{(i)}$$
2. $$\boldsymbol{\sigma}_B{}^2 = \frac{1}{m_B}\sum_{i=1}^{m_B}\left(\mathbf{x}^{(i)} - \boldsymbol{\mu}_B\right)^2$$
3. $$\widehat{\mathbf{x}}^{(i)} = \frac{\mathbf{x}^{(i)} - \boldsymbol{\mu}_B}{\sqrt{\boldsymbol{\sigma}_B{}^2 + \epsilon}}$$
4. $$\mathbf{z}^{(i)} = \boldsymbol{\gamma} \otimes \widehat{\mathbf{x}}^{(i)} + \boldsymbol{\beta}$$

Learning rate scheduler

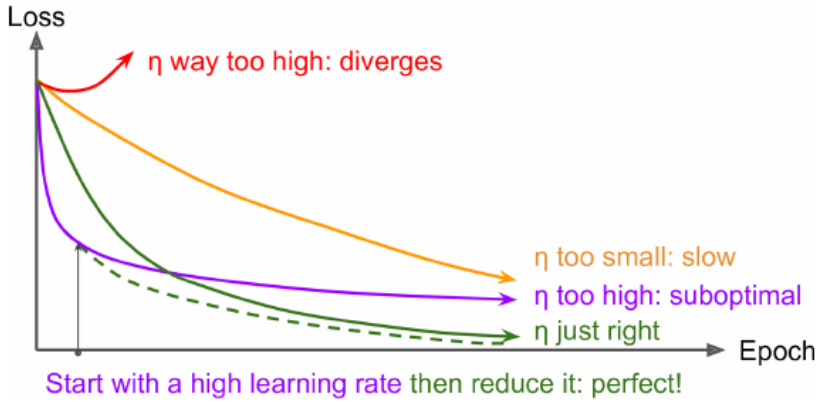
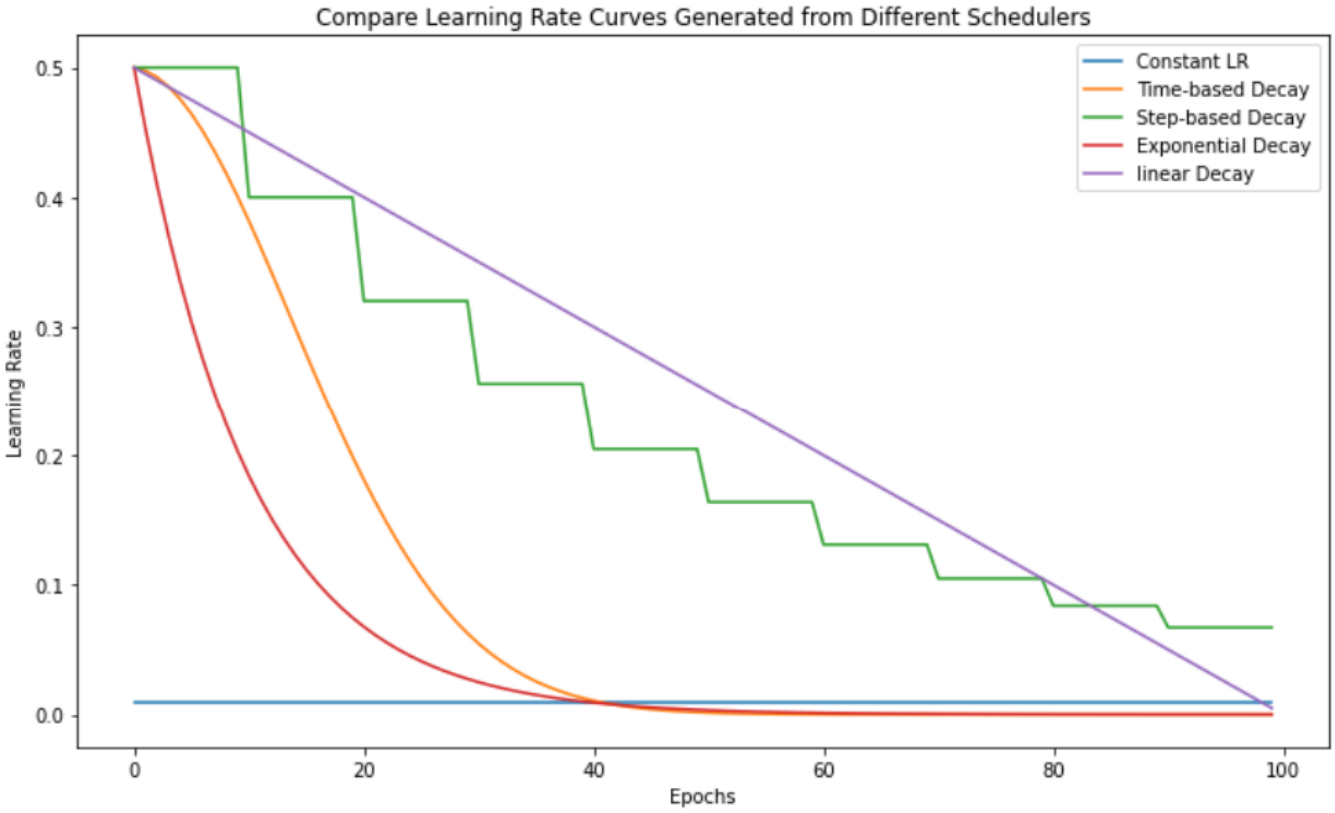


Figure 11-8. Learning curves for various learning rates η

Learning rate schedulers:

- Se incepe cu learning rate mare dupa care se scade
- sau se incepe cu lr mic, se creste, dupa care se scade din nou

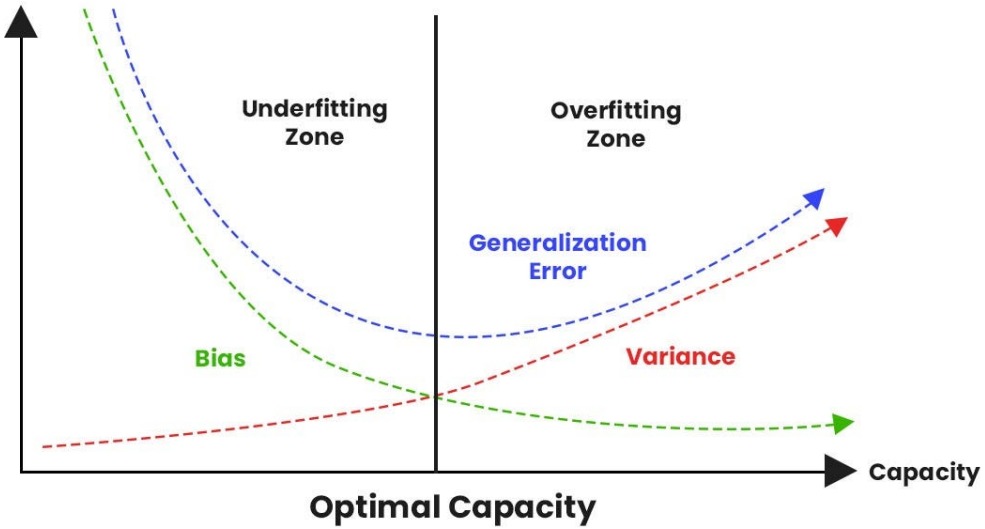
Learning rate scheduler - exemple



Exponential decay: $lr(t) = lr(0) * decay\_rate^{\frac{t}{decay\_step}}$ - reduce lr continuu, la fiecare $decay\_step$ pasi lr ajunge sa fie redus cu $decay\_rate$(e.g. 0.96)

Power decay - $lr(t) = \frac{lr(0)}{(1+t/decay\_step)^c}$; c in mod tipic e 1; la fiecare $decay_step$ pasi se reduce lr, reducerea tot scade; numita si Inverse TimeDecay

Evitare overfitting - l1, l2 regularization



L1 Regularization

$$\text{Modified loss function} = \text{Loss function} + \lambda \sum_{i=1}^{n} |W_i|$$

L2 Regularization

$$\text{Modified loss function} = \text{Loss function} + \lambda \sum_{i=1}^{n} W_i^2$$

Evitare overfitting - Dropout

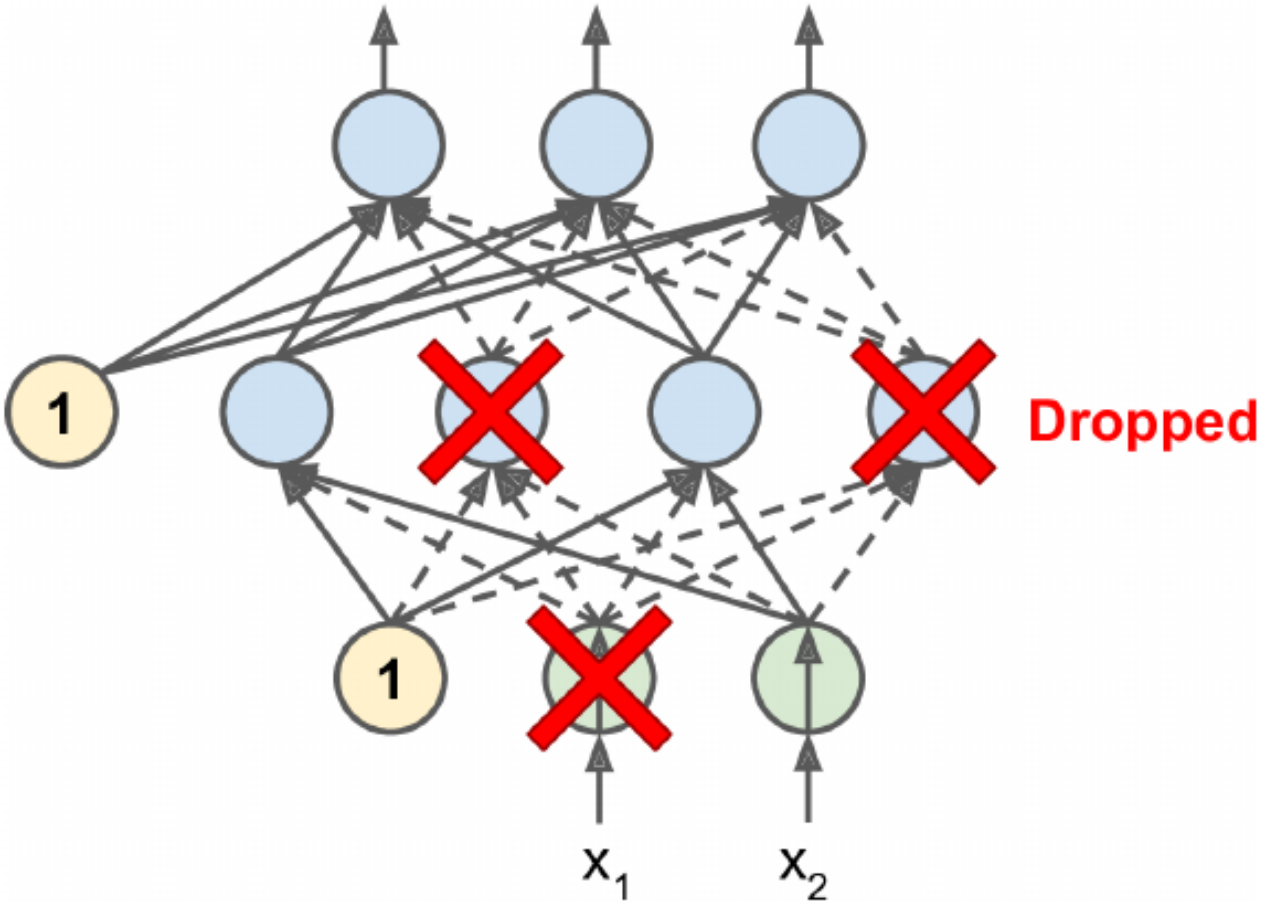


Figure 11-9. With dropout regularization, at each training iteration a random subset of all neurons in one or more layers—except the output layer—are "dropped out"; these neurons output 0 at this iteration (represented by the dashed arrows)

Activ doar in timpul invatarii

Poate fi fortat si la inferenta - MonteCarlo Dropout